Binary Search: Halving the Problem

The half-open interval, recursion, the lg N analysis, and a doubling test

Marcus Birkenkrahe

COSC 2230 · CS III · Augustana University

Fall 2026 · Sedgewick & Wayne, *Introduction to Programming in Java* 2e, §4.2

By the end of this session you can

1. explain why binary search needs a **sorted** array
2. work with the half-open interval **`a[lo, hi)`**: size, empty test, mid, halves
3. trace the recursive **`search()`** by hand, for a hit and a miss
4. derive **lg N** compares per search from the halving tree
5. run a doubling test and explain why the ratio is about 2

§4.2 · Binary search

Recap and bridge: same client, new `search()`

Last session

The client was fine; sequential `search()` was the problem.

$\sim 10N^2$ · **ratio 4**

1.28 million entries: more than 10 hours

Today

Same `WhiteFilter`, same `Generator`, same doubling test. Only `search()` changes.

$\sim 10\ N \lg N$ · **ratio about 2**

on a **sorted** array

Last session's checkout question: on a sorted whitelist, where do you look first, and how many compares does a miss need for N = 1,000,000?

§4.2 · Binary search · 5 min

The idea: examine the middle, discard half

Binary search

1. Keep the array **sorted**
2. Examine the middle key
3. Match: return its index
4. Larger than the key: search the lower half
5. Smaller: search the upper half

Key: oscar · 15 sorted names

mid	entry	oscar is	keep
7	eve	after	8..14
11	peggy	before	8..10
9	mallory	after	10..10
10	oscar	match	return 10

Four compares instead of eleven.

Watch it animated: `birkenkrahe.github.io/demos/binary-search-arithmetic.html` (panel 1)

Book p. 538: the dictionary analogy · 5 min

Binary search arithmetic: the half-open interval `a[lo, hi)`

`hi - lo`

entries: **`a[lo]`** to **`a[hi-1]`**
empty when **`hi <= lo`**

`mid = lo + (hi - lo) / 2`

half the size, added to the start; integer division rounds down

lower half: `a[lo, mid)`

stops before `mid`

upper half: `a[mid+1, hi)`

starts after `mid`

Companion 1 (panels 2 and 3):
`birkenkrahe.github.io/demos/binary-search-arithmetic.html`

Same convention as `for (i = lo; i < hi)` · 8 min

Why this form of mid: why not (lo + hi) / 2?

lo = 1,500,000,000 · hi = 2,000,000,000 · int tops out at 2,147,483,647

(lo + hi) / 2

−397,483,648

lo + hi overflows and wraps to a negative number

lo + (hi - lo) / 2

1,750,000,000

hi - lo stays small: correct

Book Q&A p. 559 · Overflow.java · 2 min

BinarySearch with edge-case tests

→ **Switch to editor · code-along 1**

1. Write the public **`search(key, a)`**: start with the interval **`[0, a.length)`**
2. Write **`search(key, a, lo, hi)`**: base case, **`mid`**, **`cmp`**, two recursive calls
3. Test client: all 15 hits, then **`aaron`**, **`marvin`**, **`zoe`**
4. Edge cases: the empty array and an array with one entry

12 min · file: `BinarySearch.java` · classpath: `.:stdlib.jar`

The recursive search() (Program 4.2.3, p. 539)

```
public static int search(String key, String[] a, int lo, int hi)
{
    if (hi <= lo) return -1;
    int mid = lo + (hi - lo) / 2;
    int cmp = a[mid].compareTo(key);
    if      (cmp > 0) return search(key, a, lo, mid);
    else if (cmp < 0) return search(key, a, mid+1, hi);
    else              return mid;
}
```

Base case 1

empty interval: a miss

Base case 2

`cmp == 0`: a hit

Otherwise

same array, smaller interval

Reference only: typed in the editor · public `search(key, a)` calls `search(key, a, 0, a.length)`

The comparison: compareTo() returns a sign, not −1, 0, +1

call	result	why
`"eve".compareTo("oscar")`	−10	`'e' - 'o'`
`"peggy".compareTo("oscar")`	1	`'p' - 'o'`
`"mallory".compareTo("marvin")`	−6	`'l' - 'r'`
`"oscar".compareTo("oscar")`	0	equal
`"Zebra".compareTo("apple")`	−7	`'Z'` (90) − `'a'` (97)

Test the sign, never the value

`cmp > 0`, `cmp < 0`, `cmp == 0`

`cmp == -1` would fail

for eve and oscar (−10)

Book: String API p. 332, Comparable pp. 451, 545–546 · `Cmp.java` · all values run in the editor

Recursion trace: searching for oscar

call	lo	hi	mid	a[mid]	key is	next
1	0	15	7	eve	after	(8, 15)
2	8	15	11	peggy	before	(8, 11)
3	8	11	9	mallory	after	(10, 11)
4	10	11	10	oscar	equal	return 10

Call 4 returns 10; calls 3, 2 and 1 pass the same 10 back up.

Sign note

"after" and "before" describe the key; the code's `cmp = a[mid].compareTo(key)` has the opposite sign

Companion 2: `birkenkrahe.github.io/demos/binary-search-trace.html` · 7 min

Correctness: invariant plus induction

Bridge

1. Scientific method on another real program
2. Model and predict the performance
3. Measure with the doubling test
4. Explain the difference

First: does `search()` find the key whenever there is one?

Invariant: if key is in a, it is in a[lo, hi)

- **Start:** `a[0, a.length)` is the whole array
- **Step:** `a[mid] > key` and **sorted**: everything from `a[mid]` to `a[hi-1]` is larger, so key is in `a[lo, mid)`. Same for `a[mid] < key` with `a[mid+1, hi)`
- **Base cases:** `cmp == 0`: return `mid`. Empty interval: key is not in a, return -1

Induction on the size `hi - lo`

Every call gets a strictly smaller interval, so a base case is always reached; `return search(...)` passes the right answer back up.

Book §4.2: the same two facts for twenty questions · 3 min

Correctness: three pieces of a proof

piece	what it guarantees	in `search()`
invariant	nothing is lost on the way	if `key` is in `a`, it is in `a[lo, hi)`
termination	an answer is always reached	the interval shrinks with every call
base cases (exit)	the answer returned is right	hit: `mid`; empty interval: -1, by the invariant

The invariant never returns anything

Only the base cases do, so they need their own check. The empty interval means "not in `a`" only because of the invariant.

Comes back: `BinarySearchLoop` (loop invariant), insertion sort (`a[0, i)` is sorted), mergesort (induction on the size).

1 min · the same table for every algorithm we prove

Correctness: each bug breaks one piece of the proof

bug	what fails
unsorted array (we run this on slide 17)	the **invariant**: the dropped half may hold the key
upper half `a[mid, hi)` (companion 1)	**termination**: with one entry left, the interval does not shrink
`(lo + hi) / 2` overflows	the **step**: `mid` lands outside `a[lo, hi)`
`while (lo <= hi)` in the loop (practice)	the **interval**: reads `a[hi]`, which is outside it

Running time: the recurrence

$T(n) =$ compares for an interval of size n: one compare, then one call on half the interval.

$$T(n) = T(n/2) + 1, \qquad T(1) = 0$$

Telescope for $n = 2^k$:

$$\begin{aligned} T(2^k) &= T(2^{k-1}) + 1 \\ &= T(2^{k-2}) + 2 \\ &= T(2^{k-3}) + 3 \\ &\vdots \\ &= T(2^0) + k = T(1) + k = k \end{aligned}$$

Result

$T(n) = k = \lg n$

Our search(), $N = 2^n - 1$

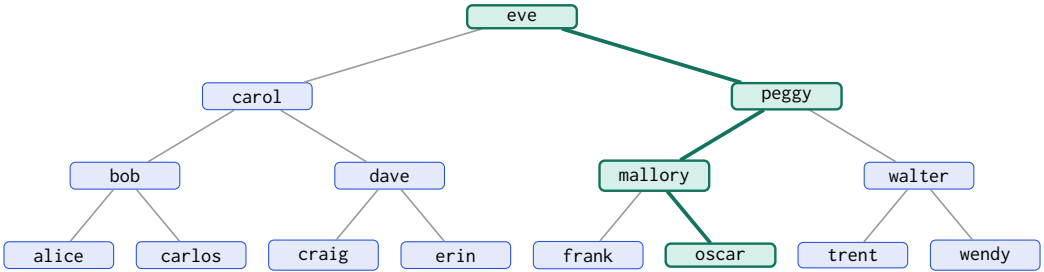
$$\begin{aligned} T(2^n - 1) &= T(2^{n-1} - 1) + 1 = \ldots \\ &= T(0) + n = n = \lg(N + 1) \end{aligned}$$

$T(0) = 0$: the empty interval, a miss

Next: the same result as a picture, one compare per level.

Book p. 535 · twenty questions: $T(1) = 0$, the number is always there · 2 min

The halving tree for $N = 2^n - 1$



Interval sizes: **15** $\to$ **7** $\to$ **3** $\to$ **1**. Every search is a top-to-bottom path (green: oscar), one compare per level.

Companion 3: `birkenkrahe.github.io/demos/binary-search-tree.html`

Cost: lg N compares per search

Miss, $N = 2^n - 1$

`n = lg(N+1)`

exactly n compares; a random hit: also ∼lg N

Any N, worst case

$\lfloor \lg N \rfloor + 1$

20 for a million, 30 for a billion

Model for the whitelist filter

`10N × lg N = 10 N lg N`

"linearithmic": doubling N slightly more than doubles the cost

Predicted doubling ratio

$2\,(1 + 1/\lg N) \approx 2.1$

for N between 100,000 and a million

Sequential search: up to a billion compares. Binary search: **30**.

Book p. 535: recurrence $T(n) = T(n/2) + 1$ · 10 min with the tree

TestBS: first unsorted, then sorted

→ **Switch to editor · code-along 2**

1. Copy **`TestSS`** to **`TestBSUnsorted`**; call **`BinarySearch.search()`**. Don't sort. **Predict:** how many keys print?
2. Count the lines with **`wc -l`** for N = 1,000 and 10,000
3. Add **`Arrays.sort(words)`** before the timer: **`TestBS`**
4. Run the doubling test, N = 100,000 . . . 1,600,000. **Predict:** a ratio a little above 2

12 min · files: `TestBSUnsorted.java` · `TestBS.java` · `Generator.java` · `BinarySearch.class` in the same folder

Our experiment: without sorting, binary search misses 99%

N = 1,000

9,907 lines

printed for 10,000 searches, all for keys that are in the array

N = 10,000

99,909 lines

printed for 100,000 searches (one line is the time)

Discarding half only works if the order is known. Checking the order costs $\mathbf{N - 1}$ compares: **sort once, then search many times.**

Book: Exercise 4.2.4, p. 560 · counts from our sandbox run of `TestBSUnsorted`

Whitelist filter with binary search: ratio $\to 2$

N	T(N) s	ratio
100,000	1	
200,000	3	
400,000	6	2
800,000	14	2.35
1,600,000	33	2.33
...		
12,800,000	264	2

$b \approx 1$

it scales: 1.6 million entries in about half a minute

2.3 > 2.1

above the model: big arrays leave the processor's cache

Sedgewick's runs · sequential: more than 10 hours for 1.28 million

Practice: binary search without recursion

PRACTICE · CLOSED BOOK · 15 MIN

1. Write **`BinarySearchLoop.java`**: **`search(String key, String[] a)`** with a **`while`** loop
2. Keep **`lo`** and **`hi`** as local variables. What is the loop condition?
3. Run the test client: loop and recursive results must **agree**
4. Include the empty array, one entry, and keys before the first and after the last

Jon Bentley asked professional programmers to do this: most solutions were not correct.

Book: Exercise 4.2.2, p. 560 · Q&A p. 559 · hint after 5 minutes

Interactive companions: try it yourself

1 · Arithmetic

Watch oscar found; move **`lo`** and **`hi`**; overflow.

`binary-search-arithmetic.html`

2 · Trace

Step through the calls with the call stack. Shuffle the array.

`binary-search-trace.html`

3 · Tree

Build the halving tree level by level; follow a search path.

`binary-search-tree.html`

`birkenkrahe.github.io/demos/`

Use in class where they appear above, or assign as review

Wrap-up: binary search scales

- Same client, new **`search()`**: **`TestBS`** sorts first
- **`a[lo, hi)`** has `hi - lo` entries; halves **`a[lo, mid)`**, **`a[mid+1, hi)`**
- **`compareTo()`**: test the sign, never -1 or 1
- Each call shrinks the interval; base cases: empty, `cmp == 0`
- A miss costs $\lg(N+1)$ compares; $10\ N \lg N$ in total, ratio about 2
- Sorted input is the premise: unsorted, $\sim$99% are missed

Next: how do we get the whitelist into sorted order? Insertion sort, then mergesort.

Checkout question

TestBS sorted the whitelist before starting the timer. Should the sorting time count as part of the filter's cost?

And: you have 15 name cards in random order. How do you put them in alphabetical order by hand?